



R u n & B u i l d

2013182021 박장호
2017184023 이소정
2019182038 조소현

R u n
&
B u i l d
I N D E X

1.프로젝트 소개

2.개발 결과

(1) 타이틀

(2) 왕국

(3) 상점

(4) 스테이지

3.주요 코드

Run & Build



1. 프로젝트 소개

3



프로젝트 명 : RunAndBuild

작업 인원 : 3명(프로그래머3)

작업 기간 : 3개월

프로젝트 설명 : 쿠키런 킹덤의 일부 컨텐츠 모작

Youtube : <https://youtu.be/Bv4bDiOR-AE>

Github : <https://github.com/jangho-park-dev/MyRunAndBuild>



메커니즘



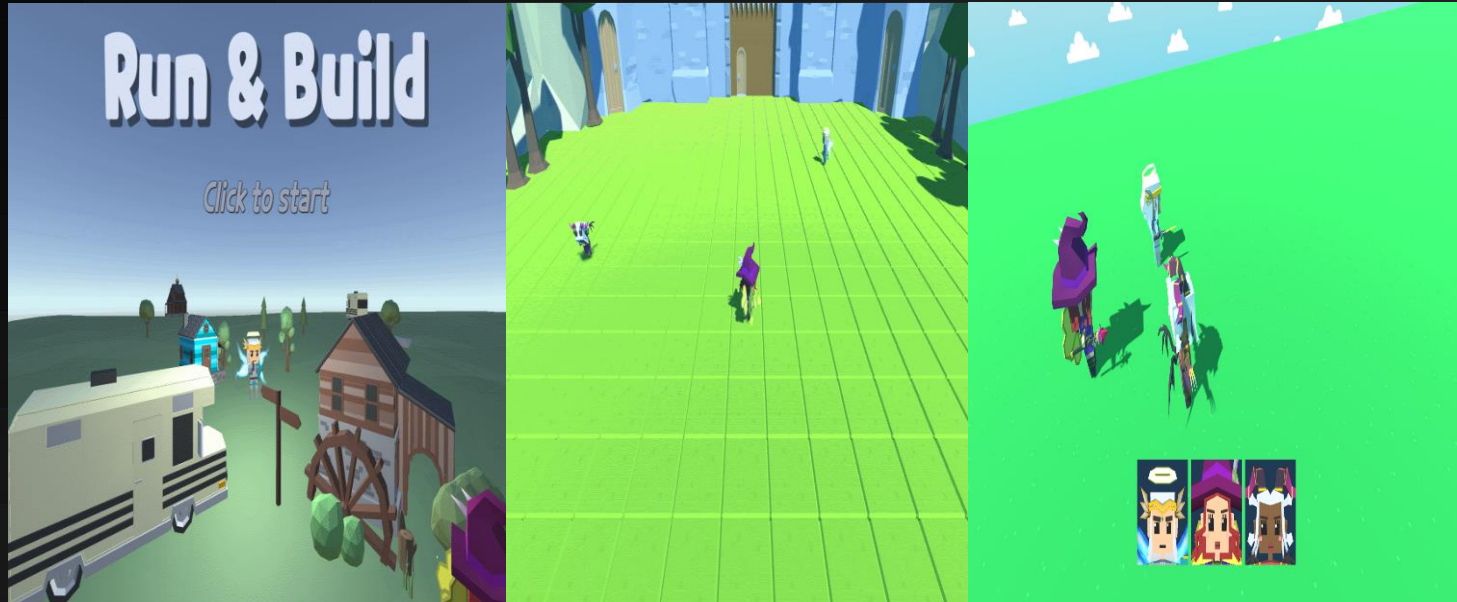
기본 메커니즘은 위와 같습니다.

Run & Build



1. 프로젝트 소개

5



조작법



모든 조작은 버튼 UI를 활용한 마우스 클릭으로 이루어집니다.

Run & Build



2. 개발 결과

6



타이틀



3D 오브젝트를 사용해서 타이틀 씬을 구성했고,
Click to start 버튼을 통해서 씬 이동 구현했습니다.

Run & Build

2. 개발 결과

7



상점 프리팹

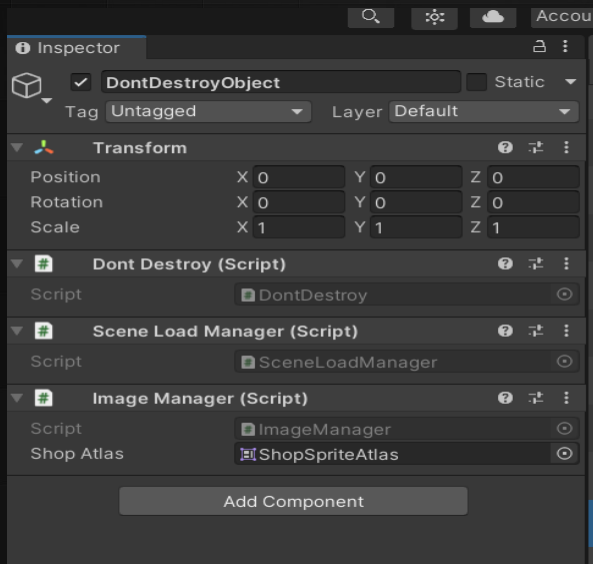


상점 버튼 기능을 구현하여 버튼을 눌렀을 때,
해당 카테고리 내용으로 replace됩니다.

Run & Build

2. 개발 결과

8



싱글톤 매니저

싱글톤 구조를 활용하여 SceneLoadManager, ImageManager, SpawnManager, GameManager, ObjectPool을 구현했습니다.

DontDestroyObject에 포함되어 있기 때문에 필요한 씬에서 SceneLoadManager, ImageManager를 활용할 수 있도록 구현했습니다.

Run & Build



2. 개발 결과

9



왕국 맵 제작



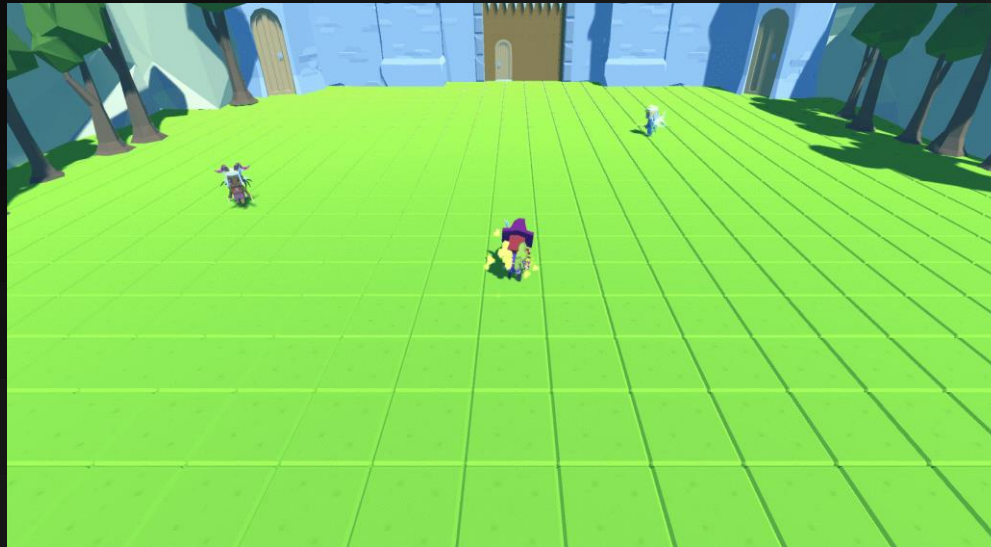
유니티 무료 에셋으로 왕국을 제작하였습니다.

Run & Build



2. 개발 결과

10



캐릭터 자유 행동



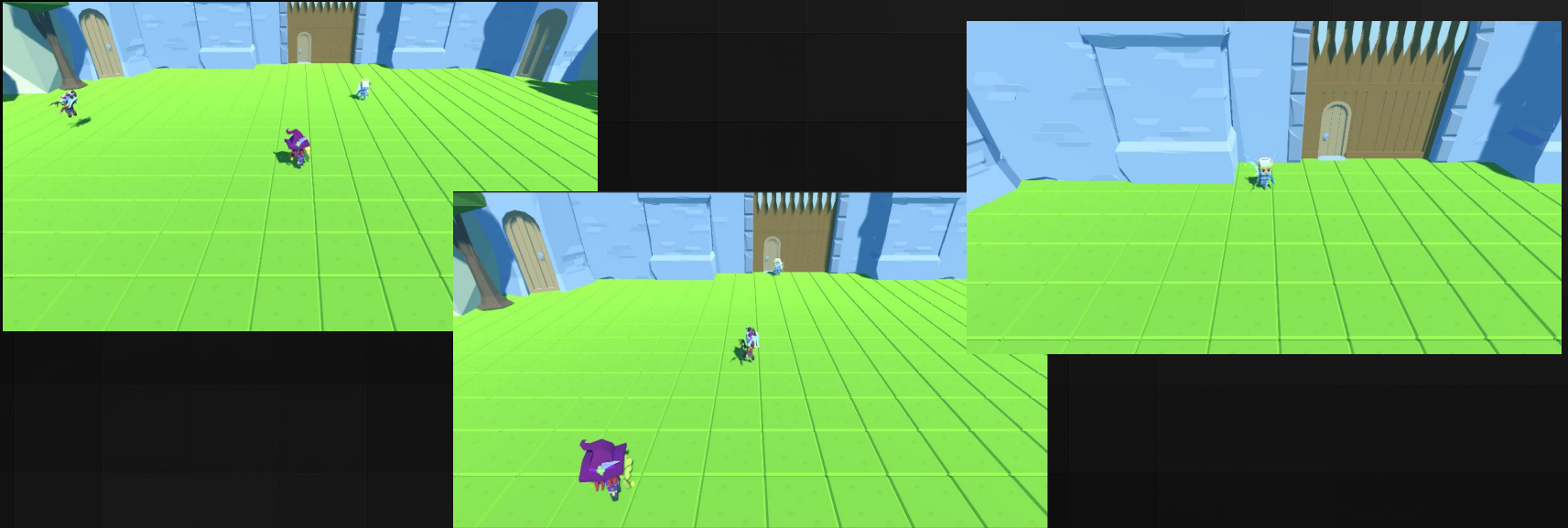
왕국의 캐릭터들이 랜덤하게 애니메이션을 전환하고,
그에 맞는 이동을 하도록 구현했습니다.

Run & Build



2. 개발 결과

11



카메라 타겟 전환



왕국의 캐릭터들을 tab키를 통해 카메라를 전환하여 바라볼 수 있습니다.



스태이지 게임 화면



적 캐릭터는 조건에 따라 스폰되어 플레이어에게 다가옵니다.

나무들은 일정 시간마다 스폰되어 지나갑니다.

플레이어와 적 캐릭터는 일정 시간마다 자동 공격합니다.

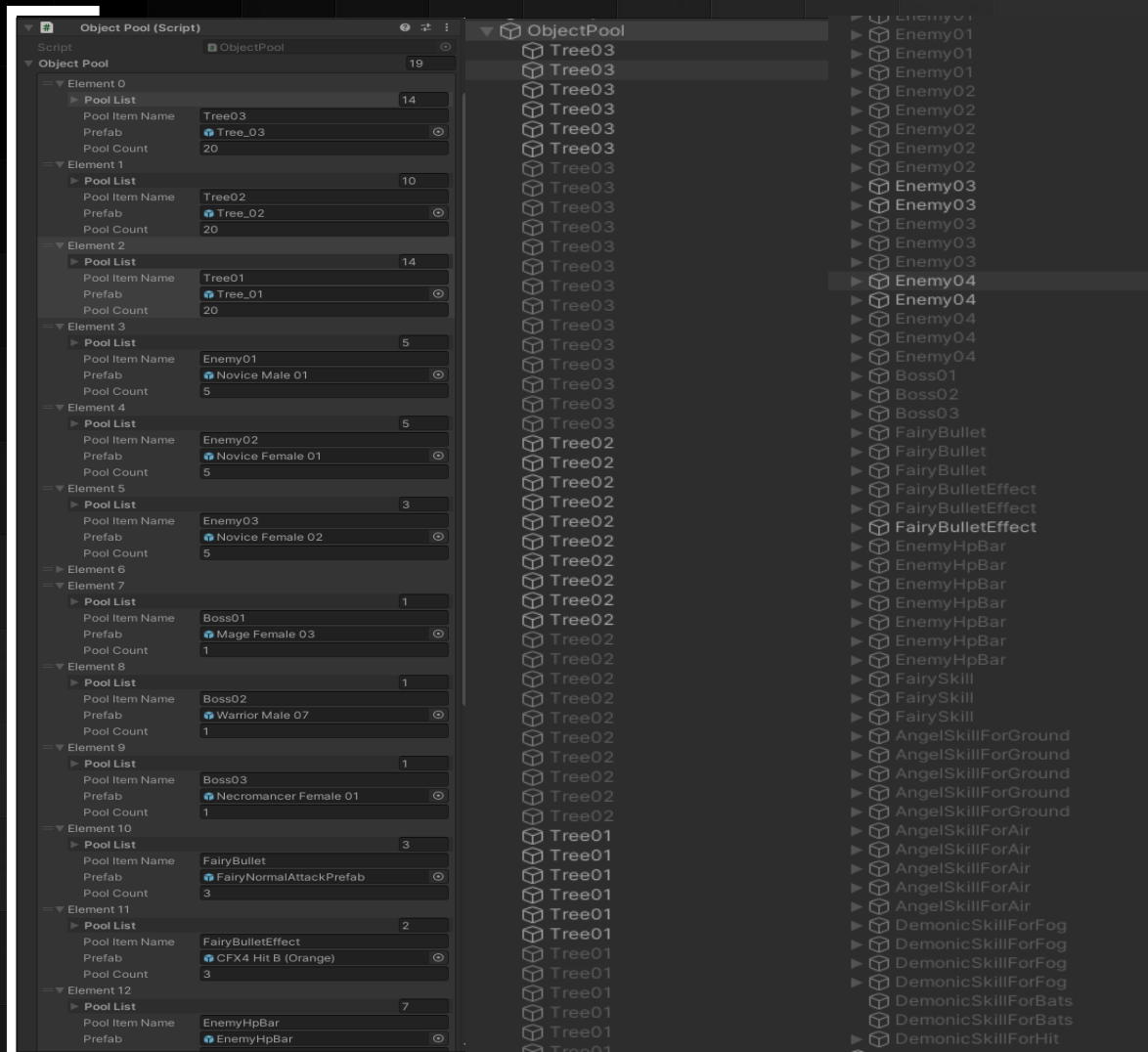
디버그에 필요한 UI는 미리 일부 구현해 놓았습니다.

Run & Build



2. 개발 결과

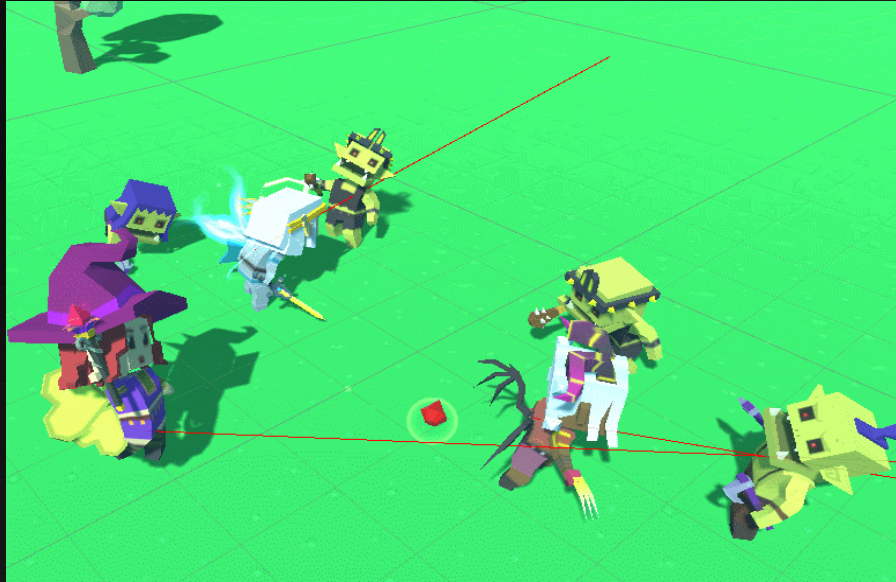
13



오브젝트 풀링



스테이지에 등장하는 객체를 효율적으로 관리했습니다.



레이 캐스트



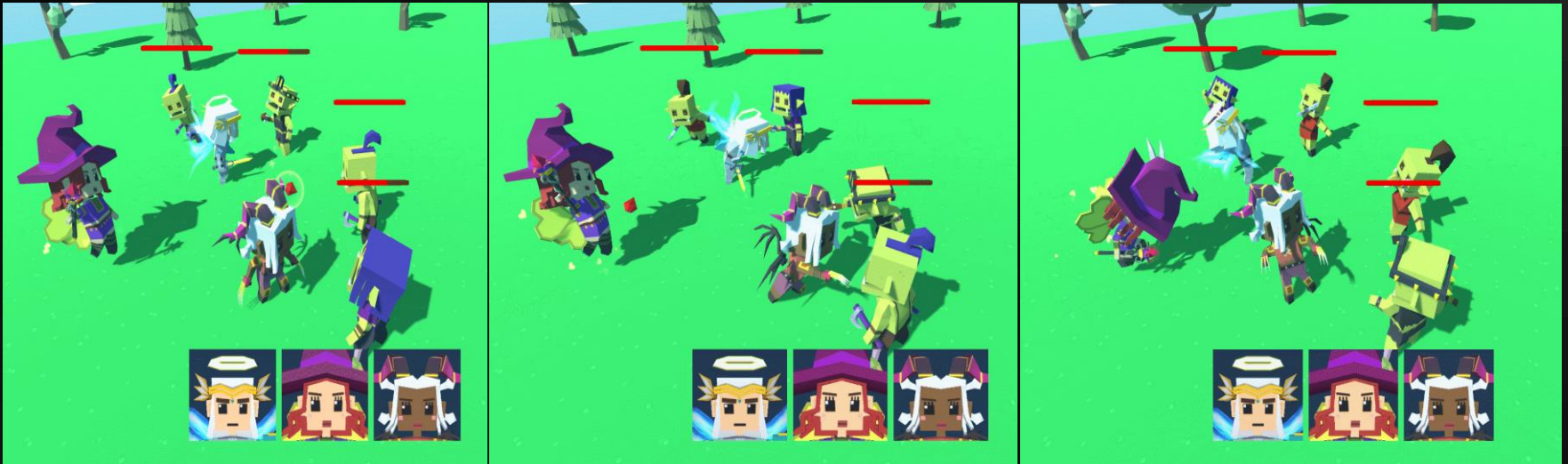
플레이어가 자동 공격할 때 앞의 적을 인식할 수 있도록
레이 캐스트 및 레이어 마스크를 이용해 탐지했습니다.

Run & Build



2. 개발 결과

15



캐릭터 스킬



각 캐릭터의 고유 스킬을 구현했습니다.

Run & Build



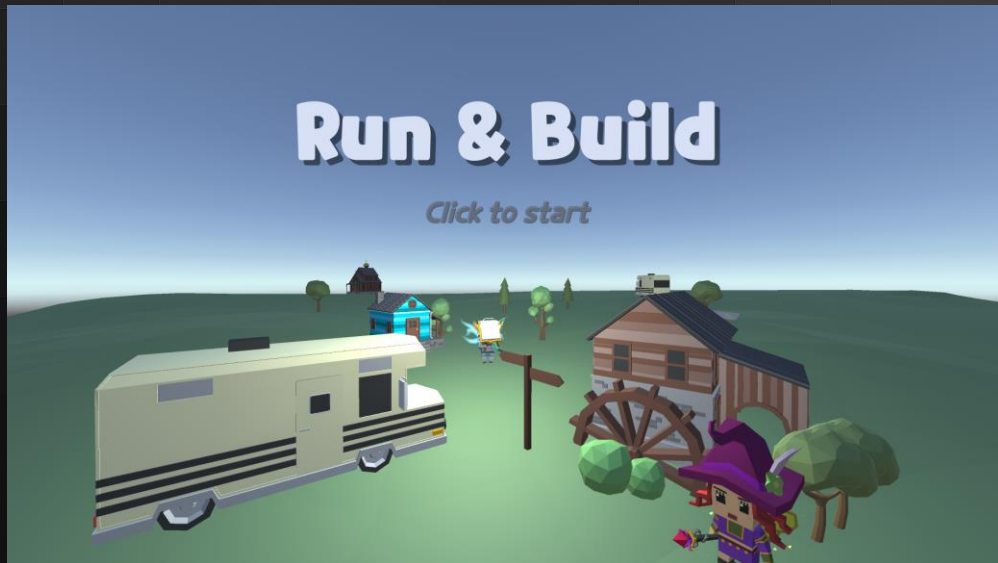
2. 개발 결과

16

(1) 타이틀

조소현

- ImageManager 구현
- SceneLoadManger 구현
- DataManager 구현
- 로그인 기능 구현
- 타이틀 씬 배치 구현



타이틀 씬 & SceneLoad

3D 오브젝트와 2D UI를 통하여 타이틀 씬을 개발하였습니다.
SceneLoadManager를 구현하여 게임 내에 씬 관리와 이동을
개발하였습니다.

Run & Build

2. 개발 결과

17

(1) 타이틀

조소현

- ImageManager 구현
- SceneLoadManger 구현
- DataManager 구현
- 로그인 기능 구현
- 타이틀 씬 배치 구현



로그인 기능 & DataManager

Title씬에서 로그인 기능을 추가하였습니다.
DataManager를 구현하여 로컬에 유저 데이터, 상품 데이터,
게임 결과 등 게임에 필요한 정보들을 저장합니다.

Run & Build



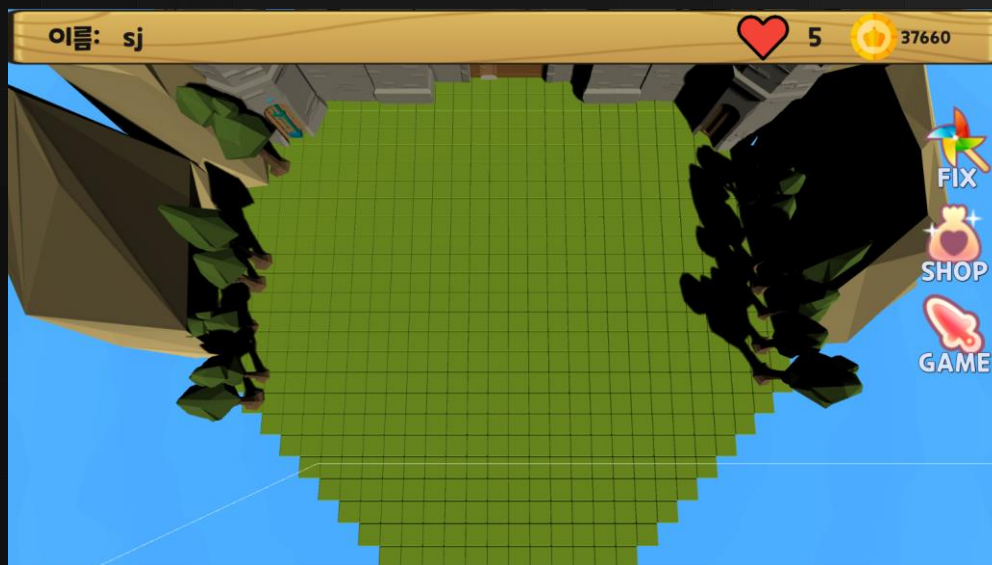
2. 개발 결과

18

(2) 왕국

이소정

- 상점 UI와 연동하여 건물 피킹 및 배치 구현
- 건물 배치 후 발생하는 이펙트 구현
- 건물 배치 시 건물 material 색 조정
- 건물 배치 후 캐릭터 위치 건물 없는 곳으로 변경
- 배치된 건물 수정 가능하도록 구현



건물 배치 및 수정



건물을 배치하고 수정하기 용이하도록 카메라를 위에서 비추도록 하였으며, 맵 위에 캐릭터들의 Active를 비활성화 하였습니다.

Run & Build



2. 개발 결과

19

(2) 왕국

이소정

- 상점 UI와 연동하여 건물 피킹 및 배치 구현
- 건물 배치 후 발생하는 이펙트 구현
- 건물 배치 시 건물 material 색 조정
- 건물 배치 후 캐릭터 위치 건물 없는 곳으로 변경
- 배치된 건물 수정 가능하도록 구현



건물 배치 및 수정

SHOP 버튼으로 건물을 배치할 수 있도록 구현하였습니다.

건물은 마우스를 따라다니며, 마우스 피킹을 이용하여 원하는 위치에 배치합니다. 마

우스 휠을 이용하여 건물을 회전시킬 수 있습니다.

FIX 버튼을 누르면 이미 배치한 건물의 위치나 방향을 수정할 수 있습니다.

Run & Build

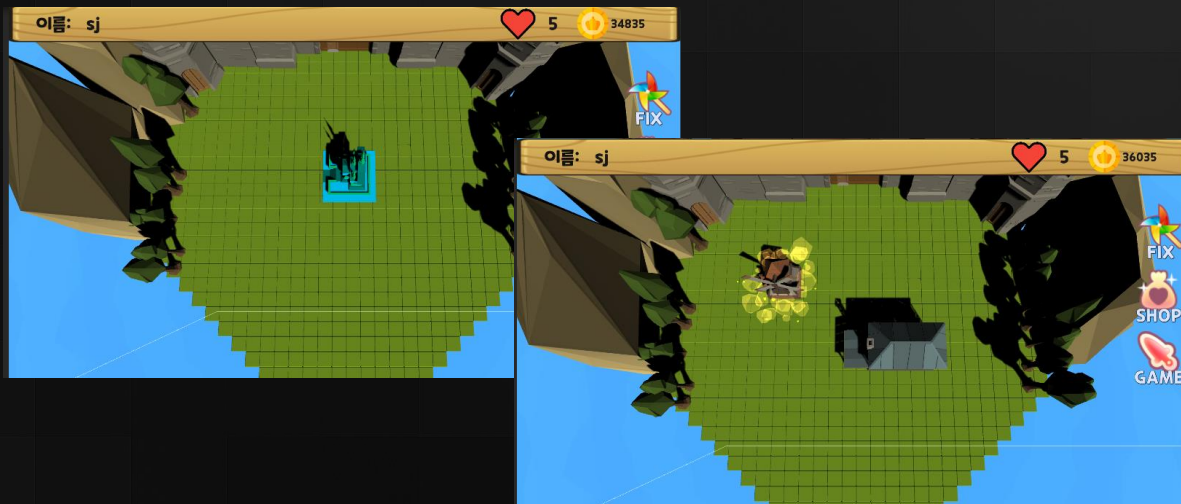
2. 개발 결과

20

(2) 왕국

이소정

- 상점 UI와 연동하여 건물 피킹 및 배치 구현
- 건물 배치 후 발생하는 이펙트 구현
- 건물 배치 시 건물 material 색 조정
- 건물 배치 후 캐릭터 위치 건물 없는 곳으로 변경
- 배치된 건물 수정 가능하도록 구현



건물 짓기 이펙트

건물 배치 시 에셋의 material을 조정하여 배치 및 수정 중인 오브젝트를 전체적으로 파랗게 하였으며, 최종 배치 시 먼지 이펙트가 발생하도록 했습니다.

Run & Build

2. 개발 결과

21

(2) 왕국

이소정

- 상점 UI와 연동하여 건물 피킹 및 배치 구현
- 건물 배치 후 발생하는 이펙트 구현
- 건물 배치 시 건물 material 색 조정
- 건물 배치 후 캐릭터 위치 건물 없는 곳으로 변경
- 배치된 건물 수정 가능하도록 구현



건물 배치 후 캐릭터 위치

건물 배치 후 캐릭터를 건물이 없는 랜덤한 위치에 리스폰 되도록 하였습니다. 해당 코드 구현 후 건물에 닿을 때마다 캐릭터가 리스폰 되는 문제가 생겼으나 시간 상의 문제로 고치지 못했습니다.

Run & Build

2. 개발 결과

22

(3) 상점

조소현

- ImageManager구현
- 상점 Tab 기능 구현
- 상점 스크롤 구현
- 상점 UI 구현
- 상품 상세페이지 구현
- 상품 구매 및 저장기능 구현
- 재화 데이터 관리 구현



상점 구현

ImageManager를 구현하였고, spriteatlas를 통해 상품의 id로 이미지를 받아오도록 구현하였습니다.

또한 카테고리 별로 선택할 수 있도록, Tab기능을 구현하였으며 건물 오브젝트들이 추가되면 스크롤하여 확인할 수 있도록 구현하였습니다.

Run & Build



2. 개발 결과

23

(3) 상점

조소현

- ImageManager구현
- 상점 Tab 기능 구현
- 상점 스크롤 구현
- 상점 UI 구현
- 상품 상세페이지 구현
- 상품 구매 및 저장기능 구현
- 재화 데이터 관리 구현



상점 구현

상점 상세페이지를 구현하였습니다.

또한 앞에서 설명한 바와 같이 DataManager로 아이디를 기억하여 상품을 구매하고 저장하는 기능과 재화 데이터를 관리할 수 있도록 구현하였습니다.

Run & Build



2. 개발 결과

24

(4) 스테이지

주차 / 이름	박장호
9	- 캐릭터 관련 HP UI - 캐릭터 사망 시 처리
10	- 이미지 버튼 시계 방향 쿼터임 애니메이션
11	- 보스 행동 및 스킬 - 좌측 상단 골드 UI - 중앙 상단 게임 타이머
12	- 우측 상단 게임 진행 상황 프로그레스 바 및 일시 정지 버튼
13	- 일시 정지 버튼에 연결될 게임 재개/종료 버튼
14	- 달린 패널 UI
15	- 개발했던 내용으로 스테이지 2개 추가 - 내용 추가될 수 있음



캐릭터 관련 HP UI

캐릭터 아이콘 밑에 구현하였습니다.

Run & Build

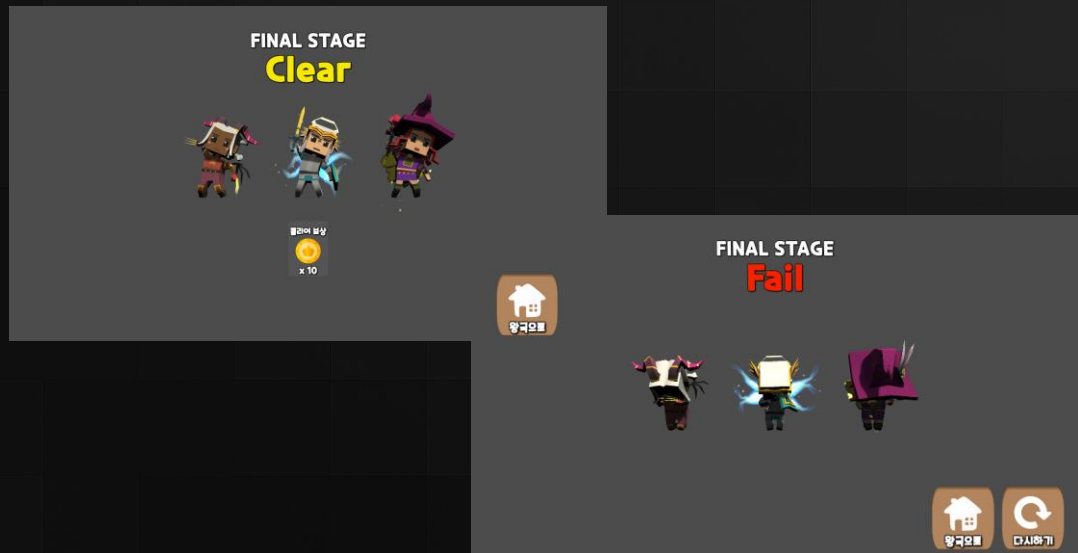


2. 개발 결과

25

(4) 스테이지

주차 / 이름	박장호
9	- 캐릭터 관련 HP UI - 캐릭터 사망 시 처리
10	- 이미지 버튼 시계 방향 쿼터임 애니메이션
11	- 보스 행동 및 스킬 - 좌측 상단 골드 UI
12	- 중앙 상단 게임 타이머
13	- 우측 상단 게임 진행 상황 프로그레스 바 및 일시 정지 버튼
14	- 일시 정지 버튼에 연결될 게임 재개/종료 버튼
15	- 개발했던 내용으로 스테이지 2개 추가 - 내용 추가될 수 있음



캐릭터 사망 시 처리

팀원 조소현이 UI로 구현하였습니다.

Run & Build



2. 개발 결과

26

(4) 스테이지

주차 / 이름	박장호
9	- 캐릭터 관련 HP UI - 캐릭터 사망 시 처리
10	- 이미지 버튼 시계 방향 쿨타임 애니메이션
11	- 보스 행동 및 스킬 - 좌측 상단 골드 UI
12	- 중앙 상단 게임 타이머 - 우측 상단 게임 진행 상황 프로그레스 바 및 일시 정지 버튼
13	- 일시 정지 버튼에 연결될 게임 재개/종료 버튼
14	- 달린 패널 UI
15	- 개발했던 내용으로 스테이지 2개 추가 - 내용 추가될 수 있음



이미지 버튼 쿨타임

이미지 버튼 쿨타임 애니메이션을 구현하였습니다.

Run & Build

2. 개발 결과

27

(4) 스테이지

주차 / 이름	박장호
9	- 캐릭터 관련 HP UI - 캐릭터 사망 시 처리
10	- 이미지 버튼 시계 방향 쿼터임 애니메이션
11	- 보스 행동 및 스킬 - 좌측 상단 골드 UI
12	- 중앙 상단 게임 타이머
13	- 우측 상단 게임 진행 상황 프로그레스 바 및 일시 정지 버튼
14	- 일시 정지 버튼에 연결될 게임 재개/종료 버튼 - 달린 패널 UI
15	- 개발했던 내용으로 스테이지 2개 추가 - 내용 추가될 수 있음



스테이지 및 보스 2개 추가 구현

스테이지는 총 3개로 구성되어 있습니다.

Run & Build



2. 개발 결과

28

(4) 스테이지

주차 / 이름	박장호
9	- 캐릭터 관련 HP UI - 캐릭터 사망 시 처리
10	- 이미지 버튼 시계 방향 쿨타임 애니메이션
11	- 보스 행동 및 스킬 - 좌측 상단 골드 UI
12	- 중앙 상단 게임 타이머 - 우측 상단 게임 진행 상황 프로그레스 바 및 일시 정지 버튼
13	- 일시 정지 버튼에 연결될 게임 재개/종료 버튼
14	- 달린 패널 UI
15	- 개발했던 내용으로 스테이지 2개 추가 - 내용 추가될 수 있음



보스 행동 및 스킬

기존 적 스크립트를 토대로 보스 행동을 구현하였습니다.

Run & Build



2. 개발 결과

29

(4) 스테이지

주차 / 이름	박장호
9	- 캐릭터 관련 HP UI - 캐릭터 사망 시 처리
10	- 이미지 버튼 시계 방향 쿼터임 애니메이션
11	- 보스 행동 및 스킬 - 좌측 상단 골드 UI
12	- 중앙 상단 게임 타이머 - 우측 상단 게임 진행 상황 프로그레스 바 및 일시 정지 버튼
13	- 일시 정지 버튼에 연결될 게임 재개/종료 버튼
14	- 달린 패널 UI
15	- 개발했던 내용으로 스테이지 2개 추가 - 내용 추가될 수 있음



보스 행동 및 스킬

기존 적 스크립트를 토대로 보스 행동을 구현하였습니다.

Run & Build



2. 개발 결과

30

(4) 스테이지

주차 / 이름	박장호
9	- 캐릭터 관련 HP UI - 캐릭터 사망 시 처리
10	- 이미지 버튼 시계 방향 쿨타임 애니메이션
11	- 보스 행동 및 스킬 - 좌측 상단 골드 UI
12	- 중앙 상단 게임 타이머 - 우측 상단 게임 진행 상황 프로그레스 바 및 일시 정지 버튼
13	- 일시 정지 버튼에 연결될 게임 재개/종료 버튼
14	- 달린 패널 UI
15	- 개발했던 내용으로 스테이지 2개 추가 - 내용 추가될 수 있음



보스 행동 및 스킬

기존 적 스크립트를 토대로 보스 행동을 구현하였습니다.

Run & Build

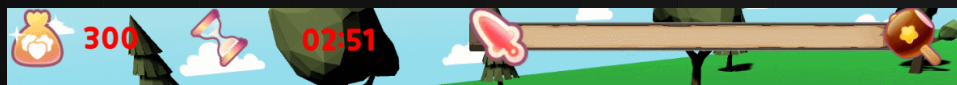


2. 개발 결과

31

(4) 스테이지

주차 / 이름	박장호
9	- 캐릭터 관련 HP UI - 캐릭터 사망 시 처리
10	- 이미지 버튼 시계 방향 쿼터임 애니메이션
11	- 보스 행동 및 스킬 - 좌측 상단 골드 UI
12	- 중앙 상단 게임 타이머
13	- 우측 상단 게임 진행 상황 프로그레스 바 및 일시 정지 버튼
14	- 일시 정지 버튼에 연결될 게임 재개/종료 버튼 달린 패널 UI
15	- 개발했던 내용으로 스테이지 2개 추가 - 내용 추가될 수 있음



상단 UI

좌측 골드 UI, 중앙 게임 시간 UI,
게임 진행 프로그레스 바 UI를 구현하였습니다.

Run & Build



2. 개발 결과

32

(4) 스테이지

주차 / 이름	박장호
9	- 캐릭터 관련 HP UI - 캐릭터 사망 시 처리
10	- 이미지 버튼 시계 방향 쿼터임 애니메이션
11	- 보스 행동 및 스킬 - 좌측 상단 골드 UI
12	- 중앙 상단 게임 타이머 - 우측 상단 게임 진행 상황 프로그레스 바 및 일시 정지 버튼
13	- 일시 정지 버튼에 연결될 게임 재개/종료 버튼
14	- 달린 패널 UI
15	- 개발했던 내용으로 스테이지 2개 추가 - 내용 추가될 수 있음



일시정지 버튼 구현

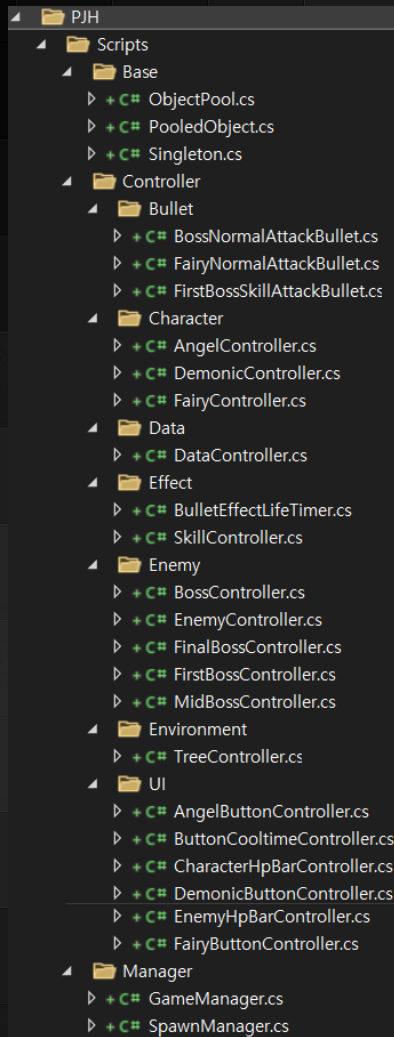
조소현 팀원이 일시정지 버튼과
계속하기/나가기 기능을 프리팹으로 구현하였습니다.

Run & Build



3. 주요 코드

33



[Scripts – 스크립트 구조]

제가 담당했던 스크립트의 구조입니다.
Base, Controller, Manager로 관리합니다.

Run & Build



3. 주요 코드

34

[Base – Singleton.cs]

Unity 스크립트 | 참조 6개

```
public class Singleton<T> : MonoBehaviour where T : MonoBehaviour
{
    private static T _instance;

    참조 99+개
    public static T Instance
    {
        get
        {
            if (_instance == null)
            {
                _instance = FindObjectOfType(typeof(T)) as T;

                if (_instance == null)
                {
                    Debug.LogError("There's no active " + typeof(T) + " in this scene");
                }
            }

            return _instance;
        }
    }
}
```

MonoBehaviour를 상속받은 타입만 사용할 수 있도록
제약 조건을 걸었습니다.

제약 조건에 맞는 여러 클래스에서 쉽게 사용할 수 있도록
상속 가능하게 만들었고, 이를 통해 ObjectPool,
DataController, SpawnManager, GameManager를 제
작하였습니다.

마찬가지로 이를 활용해 팀원이 만든 KingdomScene,
DataManager도 존재합니다.

Run & Build



3. 주요 코드

35

[Base – PooledObject.cs]

```
private GameObject CreateItem(Transform parent = null)
{
    GameObject item = Object.Instantiate(prefab) as GameObject;
    item.name = poolItemName;
    item.transform.SetParent(parent);
    item.SetActive(false);
    return item;
}
```

```
public void Initialize(Transform parent = null)
{
    for (int ix = 0; ix < poolCount; ++ix)
        poolList.Add(CreateItem(parent));
}
```

CreateItem은 prefab을 받아 오브젝트를 하나 만들고 parent에 종속 시킵니다. Parent는 ObjectPool이고 후술하겠습니다.

Initialize로 각 poolList에 오브젝트를 추가할 수 있도록 합니다.

Run & Build



3. 주요 코드

36

[Base - PooledObject.cs]

```
public void PushToPool(GameObject item, Transform parent = null)
{
    item.transform.SetParent(parent);
    item.SetActive(false);
    poolList.Add(item);
}
```

참조 1개

```
public GameObject PopFromPool(Transform parent = null)
{
    if (poolList.Count == 0)
        poolList.Add(CreateItem(parent));

    GameObject item = poolList[0];
    poolList.RemoveAt(0);
    item.SetActive(true);
    return item;
}
```

ObjectPool에서 사용할 함수들입니다.

PushToPool은 ObjectPool에 오브젝트를 종속시킨 후
poolList에 넣어줍니다.

PopFromPool은 poolList의 첫번째 오브젝트를
active시켜주고 반환합니다.

Run & Build



3. 주요 코드

37

[Base - ObjectPool.cs]

```
public class ObjectPool : Singleton<ObjectPool>
{
    public List<PooledObject> objectPool = new List<PooledObject>();

    Unity 메시지 참조 0개
    void Awake()
    {
        for (int i = 0; i < objectPool.Count; ++i)
            objectPool[i].Initialize(transform);
    }
}
```

```
PooledObject GetPoolItem(string itemName)
{
    for (int i = 0; i < objectPool.Count; ++i)
    {
        if (objectPool[i].poolItemName.Equals(itemName))
            return objectPool[i];
    }

    Debug.LogWarning("There's no matched pool list.");

    return null;
}
```

Singleton 및 PooledObject 클래스를 List 구조로 하여
ObjectPool을 관리합니다.

GetPoolItem은 오브젝트의 이름을 이용해 풀에서 탐색
합니다. 풀에 그 이름의 오브젝트가 있다면 그 오브젝트를
반환합니다.

Run & Build



3. 주요 코드

38

[Base – ObjectPool.cs]

참조 12개

```
public bool PushToPool(string itemName, GameObject item, Transform parent = null)
{
    PooledObject pool = GetPoolItem(itemName);

    if (pool == null)
        return false;

    pool.PushToPool(item, parent == null ? transform : parent);

    return true;
}
```

참조 57개

```
public GameObject PopFromPool(string itemName, Transform parent = null)
{
    PooledObject pool = GetPoolItem(itemName);

    if (pool == null)
        return null;

    return pool.PopFromPool(parent);
}
```

PushToPool은 GetPoolItem을 이용해 PooledObject에 오브젝트를 넣고, PooledObject의 PushToPool함수를 이용해 오브젝트를 추가합니다.

GetPoolItem은 오브젝트의 이름을 이용해 풀에서 탐색합니다. 풀에 그 이름의 오브젝트가 있다면 그 오브젝트를 반환합니다.

Run & Build



3. 주요 코드

39

[Manager – GameManager.cs]

```
private void InitializeCharacter()
{
    if (fc)
    {
        fc.Initialize();
        fc.transform.rotation = Quaternion.Euler(baseDirection);
        fc animator.SetBool("Fly Forward", true);
    }

    if (ac)
    {
        ac.Initialize();
        ac.transform.rotation = Quaternion.Euler(baseDirection);
        ac animator.SetBool("Run", true);
    }

    if (dc)
    {
        dc.Initialize();
        dc.transform.rotation = Quaternion.Euler(baseDirection);
        dc animator.SetBool("Run", true);
    }
}
```

```
public void Initialize()
{
    GameManager.Instance.meetEnemy = false;
    onceForCoroutine = false;
    StopCoroutine(angelAttackTimer);
}
```

InitializeCharacter()는 각 캐릭터들의 정보를 기본값으로 설정합니다.

fc, ac, dc는 각각 fairyCharacter, angelCharacter, demonicCharacter의 약자입니다.

Run & Build



3. 주요 코드

40

[Manager – GameManager.cs]

```
public int SearchEnemy()
{
    enemyArray = GameObject.FindGameObjectsWithTag("Enemy");
    int saveNumber = 0;

    if (enemyArray.Length > 0)
    {
        float minDistance = DistanceToEnemy(enemyArray[0]);
        for (int i = 1; i < enemyArray.Length; ++i)
        {
            if (minDistance >= DistanceToEnemy(enemyArray[i]))
            {
                minDistance = DistanceToEnemy(enemyArray[i]);
                saveNumber = i;
            }
        }

        return saveNumber;
    }
    else
    {
        Debug.Log("Search()'s return value is -1");
        return -1;
    }
}
```

SearchEnemy()는 플레이어가 적을 찾는 함수입니다.

GameManager가 있는 자리에 캐릭터가 가만히 있고
배경이나 적이 다가오는 로직이기 때문에, 적과의 거리를
확인해 가장 가까운 적의 인덱스 정보를 받아올 수
있습니다.

Run & Build



3. 주요 코드

41

[Manager – GameManager.cs]

```
void Update()
{
    GameTimerUpdate();

    if (meetEnemy && !onceForCoroutine)
    {
        SearchCoroutine = StartCoroutine(DistanceToEnemyTimer());
        onceForCoroutine = true;
    }

    // 눈 앞의 적이 다 죽었을 때
    if (enemyArray.Length == 0 && meetEnemy && onceForCoroutine)
    {
        StopCoroutine(SearchCoroutine);
        meetEnemy = false;
        onceForCoroutine = false;
        ++gameLevel;
        gameProgressBar.value = gameLevel * 0.33f;

        SpawnManager.Instance.MethodForStartingTreeSpawn();
        SpawnManager.Instance.MethodForStartingMonsterSpawn();
        SpawnManager.Instance.treeSpawnFlag = true;

        InitializeCharacter();
    }
}
```

```
IEnumerator DistanceToEnemyTimer()
{
    SearchEnemy();
    yield return new WaitForSeconds(0.25f);
}
```

Update() 함수에서 적 탐색 및 적 전멸 검사, 전멸했으면 SpawnManager의 함수들을 호출하고 캐릭터들을 원위치 시킵니다.

Run & Build



3. 주요 코드

42

[Manager – SpawnManager.cs]

```
// 오브젝트가 출현할 위치를 담을 배열
public Transform[] spawnPoints;

@ Unity 메시지 | 참조 0개
void Start()
{
    // Hierarchy View의 Spawn Point를 찾아 하위에 있는 모든 Transform 컴포넌트를 찾아옴
    spawnPoints = GameObject.Find("SpawnPoint").GetComponentsInChildren<Transform>();
    treeSpawningCoroutine = StartCoroutine(StartTreeSpawning());
    monsterSpawningCoroutine = StartCoroutine(StartMonsterSpawning());

    treeSpawnFlag = true;

    treeSpawnOffSet = new Vector3(0f, 1f, 0f);
    enemySpawnOffSet = new Vector3(0f, 1.21f, 0f);
}
```

Hierarchy창에 SpawnPoint를 미리 만들어두고, 그 위치를 가져와 활용할 수 있도록 합니다.
게임이 시작되면 코루틴을 실행해 나무와 몬스터가 spawn됩니다.

Run & Build



3. 주요 코드

43

[Manager – SpawnManager.cs]

```
public IEnumerator StartTreeSpawning()
{
    while (true)
    {
        foreach (var point in spawnPoints)
        {
            if (point.name.Substring(0, 4) == "tree")
            {
                int randNum = Random.Range(0, treeCount);
                switch (randNum)
                {
                    case 0:
                        treeName += "01";
                        break;
                    case 1:
                        treeName += "02";
                        break;
                    case 2:
                        treeName += "03";
                        break;
                }
                GameObject tree = ObjectPool.Instance.PopFromPool(treeName);
                tree.transform.position = point.position + treeSpawnOffset;
                treeName = "Tree";
            }
        }
        yield return new WaitForSeconds(0.5f);
    }
}
```

StartTreeSpawning()은
나무 spawn coroutine입니다.

나무가 플레이어 쪽으로 다가오는 로직이기에, spawn
point만 따로 정해놓고 그 자리에 나무를 생성합니다.

나무는 ObjectPool로 관리합니다.

Run & Build



3. 주요 코드

44

[Manager – SpawnManager.cs]

```
public IEnumerator StartMonsterSpawning()
{
    yield return new WaitForSeconds(2f);
    foreach (var point in spawnPoints)
```

```
else if (GameManager.Instance.gameLevel == 3 && point.name.Substring(0, 6) == "enemyB") // 보스
{
    switch(SceneManager.GetActiveScene().name)
    {
        case "FirstStage":
        {
            GameObject boss = ObjectPool.Instance.PopFromPool("FirstBoss");
            boss.transform.position = point.position + enemySpawnOffset;
            break;
        }
        case "MiddleStage":
        {
            GameObject boss = ObjectPool.Instance.PopFromPool("MiddleBoss");
            boss.transform.position = point.position + enemySpawnOffset;
            break;
        }
        case "FinalStage":
        {
            GameObject boss = ObjectPool.Instance.PopFromPool("FinalBoss");
            boss.transform.position = point.position + enemySpawnOffset;
            break;
        }
    }
}
```

StartMonsterSpawning()은
적 spawn coroutine입니다.

나무 spawn과 동일한 로직으로 구현해 윗부분은 생략
했습니다. 적 또한 ObjectPool로 관리합니다.

보스는 Scene 이름에 따라 다르게 spawn되도록 구현
했습니다.

Run & Build



3. 주요 코드

45

[Controller – DemonicController.cs]

```
void Update()
{
    if (GameManager.Instance.meetEnemy && !onceForCoroutine)
    {
        animator.SetBool("Run", false);
        demonicAttackTimer = StartCoroutine(PlayerAttackTimer());
        onceForCoroutine = true;
    }
}
```

참조 1개

IEnumerator PlayerAttackTimer()

```
{
    while (true)
    {
        LookAtEnemy();
        yield return new WaitForSeconds(0.5f);
        TargetToEnemy();
        yield return new WaitForSeconds(1.5f);
    }
}
```

캐릭터들은 거의 동일한 로직으로 동작하기 때문에,
DemonicController만 예시로 보여드리겠습니다.

PlayerAttackTimer()는 일정 주기로 적을 바라보거나
공격하는 것을 자동화하는 coroutine입니다.

Run & Build



3. 주요 코드

46

[Controller – DemonicController.cs]

```
private void LookAtEnemy()
{
    GameManager.Instance.enemyArray = GameObject.FindGameObjectsWithTag("Enemy");
    if (GameManager.Instance.enemyArray.Length == 0)
        return;

    int saveNumber = 0;
    float minDistance = DistanceToEnemy(GameManager.Instance.enemyArray[0]);
    for (int i = 1; i < GameManager.Instance.enemyArray.Length; ++i)
    {
        if (minDistance >= DistanceToEnemy(GameManager.Instance.enemyArray[i]))
        {
            minDistance = DistanceToEnemy(GameManager.Instance.enemyArray[i]);
            saveNumber = i;
        }
    }
    transform.LookAt(GameManager.Instance.enemyArray[saveNumber].transform);

    if (!animator.GetCurrentAnimatorStateInfo(0).IsName("Melee Left Attack 01"))
    {
        if (demonicHp > 0f)
            animator.Play("Right Punch Attack");
    }
}
```

LookAtEnemy()로 가장 가까운 적을 바라볼 수 있습니다. 이는 GameManager에 있던 SearchEnemy와 유사하게 동작하지만, 서로 의미가 다릅니다.

공격은 일정 주기로 자동 진행되기 때문에, 그에 맞춰 애니메이션이 재생될 수 있도록 설정했습니다.

Run & Build



3. 주요 코드

47

[Controller – DemonicController.cs]

```
private void TargetToEnemy()
{
    RaycastHit hit;
    if (Physics.Raycast(transform.position + new Vector3(0f, 0.75f, 0f), transform.forward, out hit, 10f, layerMask))
    {
        //Debug.DrawRay(transform.position + new Vector3(0f, 0.75f, 0f), transform.forward * 10f, Color.red, 10f);

        if (hit.collider.gameObject.name.Substring(0, 4) == "Enem")
            hit.collider.gameObject.GetComponent<EnemyController>().TakeDamage(demonicPower);
        else
            hit.collider.gameObject.GetComponent<BossController>().TakeDamage(demonicPower);
    }
}
```

캐릭터 앞을 레이 캐스팅하여 앞에 적이 있는지 확인하고,
적이 보스인지 아닌지에 따라 적용 오브젝트가 달라지도록 합니다.

Run & Build



3. 주요 코드

48

[Controller – DemonicController.cs]

```
public void TakeDamage(float damage)
{
    demonicHp -= damage;

    GameManager.Instance.dHpBarImage.fillAmount = demonicHp / maxHp;

    CheckToDead();
}
```

참조 1개

```
private void CheckToDead()
{
    if (demonicHp <= 0f)
    {
        animator.Play("Die");
        demonicDeadTimer = StartCoroutine(CheckToDeadTimer());
    }
}
```

참조 1개

```
IEnumerator CheckToDeadTimer()
{
    yield return new WaitForSeconds(1.0f);

    GameManager.Instance.dBtn.gameObject.SetActive(false);
    Destroy(gameObject);
}
```

LookAtEnemy()로 가장 가까운 적을 바라볼 수 있습니다. 이는 GameManager에 있던 SearchEnemy와 유사하게 동작하지만, 서로 의미가 다릅니다.

공격은 일정 주기로 자동 진행되기 때문에, 그에 맞춰 애니메이션이 재생될 수 있도록 설정했습니다.

Run & Build



3. 주요 코드

49

[UI – DemonicButtonController.cs]

```
public void OnPointerUp(PointerEventData eventData)
{
    if (btn.enabled)
    {
        if (!GameManager.Instance.meetEnemy)
        {
            GameManager.Instance.dc animator.SetBool("Run", false);
            GameManager.Instance.dc animator.Play("Melee Left Attack 01");
            demonicSkillCoroutine = StartCoroutine(DemonicSkill());
            GameManager.Instance.dc animator.SetBool("Run", true);
        }
        else
        {
            GameManager.Instance.dc animator.Play("Melee Left Attack 01");
            demonicSkillCoroutine = StartCoroutine(DemonicSkill());
        }
    }
}
```

Demonic Character의 버튼을 누르면 호출되는 함수입니다.

GameManager의 인스턴스 변수인 meetEnemy를 통해 적과 조우했는지 아닌지를 확인하고, 애니메이션을 바꿔주는 작업을 진행합니다.

Run & Build



3. 주요 코드

50

IEnumerator DemonicSkill()

```
{  
    ObjectPool.Instance.PopFromPool("DemonicSkillForFog").transform.position = GameManager.Instance.dc.transform.position;  
  
    Vector3 dir = GameManager.Instance.fc.transform.position - enemyArray[saveNumber].transform.position;  
    dir.Normalize();  
    GameManager.Instance.dc.transform.position = enemyArray[saveNumber].transform.position - dir * 2f;  
    GameManager.Instance.dc.transform.LookAt(enemyArray[saveNumber].transform);  
  
    ObjectPool.Instance.PopFromPool("DemonicSkillForFog").transform.position = GameManager.Instance.dc.transform.position;  
    ObjectPool.Instance.PopFromPool("DemonicSkillForBats").transform.position = GameManager.Instance.dc.transform.position;  
  
    yield return new WaitForSeconds(0.5f);  
  
    ObjectPool.Instance.PopFromPool("DemonicSkillForHit").transform.position = enemyArray[saveNumber].transform.position + new Vector3(0f, 1f, 0f);  
    SkillEffect(saveNumber);  
  
    yield return new WaitForSeconds(0.5f);  
    ObjectPool.Instance.PopFromPool("DemonicSkillForFog").transform.position = GameManager.Instance.dc.transform.position;  
  
    GameManager.Instance.dc.transform.position = GameManager.Instance.dc.basePosition;  
    GameManager.Instance.dc.transform.rotation = Quaternion.Euler(new Vector3(0, 0, 1));  
  
    ObjectPool.Instance.PopFromPool("DemonicSkillForFog").transform.position = GameManager.Instance.dc.transform.position;  
    ObjectPool.Instance.PopFromPool("DemonicSkillForBats").transform.position = GameManager.Instance.dc.transform.position;  
}
```

DemonicSkill()에서는 적의 위치를 통해 캐릭터의 방향을 설정하고, 특히 demonic같은 경우 적의 배후로 순간 이동하여 찌르는 스킬을 가지고 있기 때문에 0.5초 간격으로 안개와 박쥐 이펙트가 나오도록 진행했습니다.

Run & Build



3. 주요 코드

51

[UI – ButtonCooltimeController.cs]

```
public void OnPointerUp(PointerEventData eventData)
{
    if (btn.enabled)
    {
        if (btn)
            btn.enabled = false;

        if (btn.name.Substring(0, 5) == "Angel")
        {
            print(btn.name.Substring(0, 5));
            leftTime = angelCooltime;
            CooltimeCoroutine = StartCoroutine(StartCooltime(angelCooltime));
        }
        else if (btn.name.Substring(0, 5) == "Demon")
        {
            print(btn.name.Substring(0, 5));
            leftTime = demonCooltime;
            CooltimeCoroutine = StartCoroutine(StartCooltime(demonCooltime));
        }
        else if (btn.name.Substring(0, 5) == "Fairy")
        {
            print(btn.name.Substring(0, 5));
            leftTime = fairyCooltime;
            CooltimeCoroutine = StartCoroutine(StartCooltime(fairyCooltime));
        }
    }
}
```

캐릭터의 각 스킬 버튼을 누르면 호출되는 함수입니다.

버튼을 누르면 버튼의 이름에 따라 버튼을 잠시 비활성화 시키고, 남은 시간을 버튼에 출력하게 합니다.

Run & Build



3. 주요 코드

52

[UI – ButtonCooltimeController.cs]

```
IEnumerator StartCooltime(float cooltime)
{
    while (leftTime > 0f)
    {
        leftTime -= Time.deltaTime;
        if (leftTime < 0f)
        {
            leftTime = 0f;
            if (btn)
                btn.enabled = true;
        }

        float ratio = 1.0f - (leftTime / cooltime);
        if (img)
            img.fillAmount = ratio;
        if (txt)
            txt.text = string.Format("{0:0.0#}", leftTime);

        yield return null;
    }
    if (txt)
        txt.text = "";
}
```

쿨타임을 재기 시작하는 코루틴을 위한 IEnumerator입니다.

남은 시간에 따라 원형의 fillAmount를 시각적으로 보여주고, 버튼 중앙에 쿨타임을 출력합니다. 쿨타임이 끝나면 버튼을 활성화 시킵니다.

Run & Build



3. 주요 코드

53

[Enemy – EnemyController.cs]

```
private void MoveToPlayer()
{
    transform.LookAt(GameManager.Instance.fc.transform);
    transform.position += dir * enemySpeed * Time.deltaTime;
}
```

참조 5개

```
public void TakeDamage(float damage)
{
    enemyHp -= damage;

    if (enemyHp > 0f)
        animator.Play("Take Damage");

    hpBarImage.fillAmount = enemyHp / MAX_HP;

    CheckToDead();
}
```

참조 1개

```
private void CheckToDead()
{
    if (enemyHp <= 0f)
    {
        animator.Play("Die");
        enemyDeadTimer = StartCoroutine(CheckToDeadTimer());
    }
}
```

기본적으로 적이 동작하게 하기 위해 만든 함수입니다.

플레이어 캐릭터를 향해 이동하거나, 데미지를 입거나,
데미지를 입고 죽었는지 확인할 수 있습니다.

Run & Build



3. 주요 코드

54

Enemy – EnemyController.cs

```
IEnumerator CheckToDeadTimer()
{
    yield return new WaitForSeconds(1.0f);

    DataManager.Instance.UserData.money += 100;
    GameManager.Instance.gameMoney += 100;
    GameManager.Instance.moneyText.text = GameManager.Instance.gameMoney.ToString();
    ObjectPool.Instance.PushToPool(gameObject.name, gameObject);
}
```

참조 4개

```
private void DamageDistribution()
{
    if (GameManager.Instance.ac && GameManager.Instance.ac.angelHp > 0f)
    {
        GameManager.Instance.ac.TakeDamage(enemyPower);
    }
    else if (GameManager.Instance.dc && GameManager.Instance.dc.demonicHp > 0f)
    {
        GameManager.Instance.dc.TakeDamage(enemyPower);
    }
    else if (GameManager.Instance.fc && GameManager.Instance.fc.fairyHp > 0f)
    {
        GameManager.Instance.fc.TakeDamage(enemyPower);
    }
}
```

CheckToDeadTimer()는 적이 쓰러지고 사라지는 데
까지 재생되는 애니메이션 시간 확보를 위해 만들었습
니다.

DamageDistribution() 함수는 적이 플레이어가 배치
한 캐릭터 순서대로 공격할 수 있게 하기 위해 만든 함
수입니다.

Run & Build



3. 주요 코드

55

Enemy – EnemyController.cs

```
private void SetHpBar()
{
    hb = ObjectPool.Instance.PopFromPool("EnemyHpBar");
    hb.transform.parent = GameManager.Instance.uiCanvas.GetComponent<Transform>();
    hpBarImage = hb.GetComponentsInChildren<Image>()[1];
    hpBarImage.fillAmount = MAX_HP;

    var hpBar = hb.GetComponent<EnemyHpBarController>();
    hpBar.targetTransform = gameObject.transform;
    hpBar.offset = hpBarOffset;
}
```

```
private void OnEnable()
{
    animator.SetBool("Walk", false);
    animator.SetBool("Run", true);

    // 재사용 시 초기화되어야 할 것들
    dir = new Vector3(0f, 0f, -1f);
    transform.Rotate(new Vector3(0f, 180f, 0f));
    meetPlayer = false;
    attackPlayer = false;
    onceForCoroutine = false;
    enemyHp = MAX_HP;
    enemySpeed = SPEED;

    SetHpBar();
}
```

SetHpBar() 함수는 적의 Hp를 확인할 수 있도록 HpBar를 설정합니다.

Enemy의 OnEnable() 함수에서 호출됩니다.

Run & Build



3. 주요 코드

56

Enemy – EnemyController.cs

```
IEnumerator EnemyAttackTimer()
{
    while (true)
    {
        if (enemyHp > 0f)
        {
            if (transform.name == "Enemy01")
            {
                animator.Play("Melee Left Attack 01");
                DamageDistribution();
            }
            else if (transform.name == "Enemy02")
            {
                animator.Play("Melee Right Attack 02");
                DamageDistribution();
            }
            else if (transform.name == "Enemy03")
            {
                animator.Play("Melee Right Attack 03");
                DamageDistribution();
            }
            else if (transform.name == "Enemy04")
            {
                animator.Play("Melee Right Attack 01");
                DamageDistribution();
            }
        }
        yield return new WaitForSeconds(2);
    }
}
```

적의 전체적인 기본 공격 루프를 위해 만든
IEnumerator입니다.

모든 적은 해당 분기에 따라 공격합니다.

Run & Build



3. 주요 코드

57

```
IEnumerator BossAttackTimer()
{
    while (true)
    {
        if (bossHp > 0f)
        {
            if (transform.name == "FirstBoss")
            {
                if (!animator.GetCurrentAnimatorStateInfo(0).IsName("Cast Spell 02"))
                {
                    animator.Play("Cast Spell 01");
                    Invoke("ShootTheBullet", 0.5f);
                    LookAtPlayer();
                }
            }
            else if (transform.name == "MiddleBoss")
            {
                if (!animator.GetCurrentAnimatorStateInfo(0).IsName("Cast Spell 02"))
                {
                    animator.Play("Cast Spell 01");
                    Invoke("ShootTheBullet", 0.5f);
                    LookAtPlayer();
                }
            }
            else if (transform.name == "FinalBoss")
            {
                if (!animator.GetCurrentAnimatorStateInfo(0).IsName("Spin Attack"))
                {
                    animator.Play("Melee Right Attack 03");
                    DamageDistribution();
                }
            }
        }
        yield return new WaitForSeconds(2);
    }
}
```

Enemy – BossController.cs

보스의 전체적인 기본 공격 루프를 위해 만든
IEnumerator입니다.

모든 적은 해당 분기에 따라 공격합니다.

Run & Build



3. 주요 코드

58

```
IEnumerator SkillTimer()
{
    while (true)
    {
        yield return new WaitForSeconds(0f);

        animator.SetBool("Spin Attack", true);

        if (GameManager.Instance.ac)
        {
            GameObject obj = ObjectPool.Instance.PopFromPool("FinalBossSkillEffect01");
            obj.transform.position = GameManager.Instance.ac.transform.position + new Vector3(0f, 0.1f, 0f);
        }
        if (GameManager.Instance.dc)
        {
            GameObject obj = ObjectPool.Instance.PopFromPool("FinalBossSkillEffect01");
            obj.transform.position = GameManager.Instance.dc.transform.position + new Vector3(0f, 0.1f, 0f);
        }
        if (GameManager.Instance.fc)
        {
            GameObject obj = ObjectPool.Instance.PopFromPool("FinalBossSkillEffect01");
            obj.transform.position = GameManager.Instance.fc.transform.position + new Vector3(0f, 0.1f, 0f);
        }

        yield return new WaitForSeconds(0.3f);

        if (GameManager.Instance.ac)
        {
            GameObject obj1 = ObjectPool.Instance.PopFromPool("FinalBossSkillEffect02");
            GameObject obj2 = ObjectPool.Instance.PopFromPool("FinalBossSkillEffect03");
            obj1.transform.position = GameManager.Instance.ac.transform.position + new Vector3(0f, 1f, 0f);
            obj2.transform.position = GameManager.Instance.ac.transform.position + new Vector3(0f, 1f, 0f);
            GameManager.Instance.ac.TakeDamage(GameManager.Instance.finalBossSkillAttackDamage);
        }
        if (GameManager.Instance.dc)
        {
            GameObject obj1 = ObjectPool.Instance.PopFromPool("FinalBossSkillEffect02");
            GameObject obj2 = ObjectPool.Instance.PopFromPool("FinalBossSkillEffect03");
            obj1.transform.position = GameManager.Instance.dc.transform.position + new Vector3(0f, 1f, 0f);
            obj2.transform.position = GameManager.Instance.dc.transform.position + new Vector3(0f, 1f, 0f);
            GameManager.Instance.dc.TakeDamage(GameManager.Instance.finalBossSkillAttackDamage);
        }
        if (GameManager.Instance.fc)
        {
            GameObject obj1 = ObjectPool.Instance.PopFromPool("FinalBossSkillEffect02");
            GameObject obj2 = ObjectPool.Instance.PopFromPool("FinalBossSkillEffect03");
            obj1.transform.position = GameManager.Instance.fc.transform.position + new Vector3(0f, 1f, 0f);
            obj2.transform.position = GameManager.Instance.fc.transform.position + new Vector3(0f, 1f, 0f);
            GameManager.Instance.fc.TakeDamage(GameManager.Instance.finalBossSkillAttackDamage);
        }

        yield return new WaitForSeconds(0.3f);

        if (GameManager.Instance.ac)
        {
            GameObject obj1 = ObjectPool.Instance.PopFromPool("FinalBossSkillEffect02");
            GameObject obj2 = ObjectPool.Instance.PopFromPool("FinalBossSkillEffect03");
        }
    }
}
```

Enemy – FinalBossController.cs

총 3개의 보스를 구현했고 보스마다 스킬의 차이가 있지만, 마지막 보스만 예시로 보여드리겠습니다.

FinalBoss는 Spin Attack 스킬을 쓰기 때문에 모든 캐릭터들이 공격받고, 이에 따른 이펙트를 풀에서 꺼내 옵니다.

0.3초 간격으로 총 4번에 걸쳐 공격하기 때문에 중간중간 딜레이가 있습니다.

Run & Build



끝

59

감사합니다.